

Graphs

BFS, DFS, Dijkstra – మూడూ "అతి చిన్న దారి" ఇస్తాయని అనిపిస్తుంది. మూడూ **వేర్వేరు ఊహల** మీద నడుస్తాయి, మరియు ఊహ విరిగినప్పుడు మూడూ **మౌనంగా** తప్పు జవాబు ఇస్తాయి.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – DFS అతి చిన్న దారి ఇచ్చేది **18% సార్లు** మాత్రమే, సగటున దారి **196% పొడవు**. రెండోది – ఒక్క ఋణాత్మక అంచు Dijkstra ని **1.2% cases** లో తప్పు చేస్తుంది. మూడోది – "visited" ని ఒక పంక్తి ఆలస్యంగా గుర్తుపెడితే queue **59.8 రెట్లు** పెద్దది, **జవాబు మాత్రం సరైనదే**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – మరియు §12 lo ప్రతి జవాబు **Floyd-Warshall** అనే స్వతంత్ర algorithm తో పోల్చబడినది.

ఈ **SERIES LO** నాలుగోసారి అదే ప్రశ్న

Deep 01 – binary search: predicate ఏకదిశగా ఉండాలి.

Deep 02 – sliding window: కిటికీ పెరిగితే కొలత ఏకదిశగా మారాలి.

Deep 04 – DP: memo key స్థితిని పూర్తిగా సూచించాలి.

Deep 05 – Dijkstra: ఒక `node` ని తీసుకున్నాక దాని దూరం ఇక మారదు.

నాలుగూ ఒకే ఆకారం: "నేను ముందుకి కదిలాక వెనక్కి రావాల్సిన అవసరం లేదు" అనే ఊహ. అదే వేగాన్ని ఇస్తుంది, మరియు అదే విరిగినప్పుడు తప్పు జవాబు ఇస్తుంది.

Dijkstra lo ఆ ఊహ **అంచు బరువులు ≥ 0** అయితేనే నిజం. §7 lo కొలుస్తాం.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Graph problems lo ఏకైక ముఖ్యమైన ప్రశ్న
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: DFS vs BFS

3. Step – రెండు సహజమైన శోధనలు
4. మొదటి విరుపు – DFS అతి చిన్న దారి ఇచ్చేది 18% సార్లు

Part 3 – రెండో విరుపు: ఋణాత్మక అంచులు

5. Step – బరువులు ఉంటే Dijkstra
6. Step – Dijkstra నడిచే ఊహ
7. రెండో విరుపు – ఒక్క ఋణాత్మక అంచు
8. Step – Bellman-Ford · మరియు ఋణాత్మక వలయాలు

Part 4 – మూడో విరుపు: సరైన జవాబు, తప్పు ఖర్చు

9. Step – "visited" ఎక్కడ గుర్తుపెట్టాలి
10. మూడో విరుపు – 59.8 రెట్లు ఎక్కువ పని, అదే జవాబు

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code
12. 16,000 graphs · Floyd-Warshall తో పోలిక · mutation testing

13. బతికిన రెండు మార్పులు
14. Follow-ups – A*, 0-1 BFS, topological sort
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Graph problems lo ఏకైక ముఖ్యమైన ప్రశ్న

Graph problems చాలా రకాలుగా అడుగుతారు – పటాలు, నగరాలు, పదాల గొలుసులు, ఆధారాలు. కానీ algorithm ఎంపిక ఒకే ఒక ప్రశ్న మీద ఆధారపడుతుంది:

అంచులకి బరువులు ఉన్నాయా? ఉంటే అవి ఋణాత్మకం కాగలవా?

అంచులు	ఏ algorithm	ఎందుకు
బరువులు లేవు (అన్నీ 1)	BFS	ఒక్కో పొరగా విస్తరిస్తుంది → మొదట చేరినదే అతి చిన్నది
బరువులు ≥ 0	Dijkstra	అతి చిన్న దూరం ఉన్నదాన్ని ముందు తీసుకోవడం సురక్షితం
బరువులు ఋణాత్మకం కావచ్చు	Bellman-Ford	ఏ ఊహ చేయదు – $n-1$ సార్లు అన్నింటినీ సడలిస్తుంది
అన్ని జతల మధ్య దూరాలు	Floyd-Warshall	n చిన్నదైతే (≤ 500) సరళమైనది

DFS ఈ పట్టికలో ఎక్కడా లేదు – మరియు అది ఉద్దేశపూర్వకం

DFS ఒక అద్భుతమైన సాధనం: వలయాలు కనుగొనడం, topological sort, connected components, backtracking.

అతి చిన్న దారి దాని పని కాదు. §4 lo కొలుస్తాం – DFS అతి చిన్న దారి ఇచ్చేది **18% సార్లు**, మరియు అది యాదృచ్ఛికంగా కాదు: DFS ఒక దారిని **చివరి వరకు** అనుసరిస్తుంది, పక్కనే ఉన్న చిన్న దారిని చూడకుండా.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
అంచులకి బరువులు ఉన్నాయా?	లేకపోతే BFS చాలు, Dijkstra అనవసరం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
బరువులు ఋణాత్మకం కాగలవా?	§7 – ఇదే Dijkstra ని చంపే ప్రశ్న
ఋణాత్మక వలయం ఉండొచ్చా?	ఉంటే "అతి చిన్న దారి" అనే ప్రశ్నకి అర్థమే లేదు
దిశ ఉందా (directed) లేదా?	Undirected lo ఒక ఋణాత్మక అంచు = వెంటనే ఒక ఋణాత్మక వలయం
ఎన్ని nodes, ఎన్ని అంచులు?	$n \leq 500$ అయితే Floyd-Warshall సులభం; పెద్దదైతే Dijkstra + heap
ఒక source నా, అన్ని జతలా?	ఇదే Bellman-Ford vs Floyd-Warshall ఎంపిక
దారి కావాలా, దూరం కావాలా?	దారి కావాలంటే <code>parent[]</code> ఉంచాలి (Deep 04 §10 యొక్క అదే విషయం)

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"**ఋణాత్మక అంచులు ఉంటే – ఋణాత్మక వలయం కూడా ఉండొచ్చా?**"

ఎందుకంటే ఈ రెండూ పూర్తిగా వేర్వేరు పరిస్థితులు:

ఋణాత్మక అంచులు, వలయం లేదు → Bellman-Ford సరైన జవాబు ఇస్తుంది.

ఋణాత్మక వలయం ఉంది → అతి చిన్న దారి **ఉనికిలో లేదు** (వలయం చుట్టూ తిరుగుతూ ఉంటే దూరం $-\infty$ కి వెళ్తుంది). సరైన జవాబు "లేదు" అని చెప్పడం.

§12 lo నా fuzz 16,000 graphs lo **1,003 ఋణాత్మక వలయాలు** కనుగొంది – మరియు వాటిని గుర్తించడమే ఒక నియమం.

Part 2 – మొదటి విరుపు: DFS vs BFS

3. Step – రెండు సహజమైన శోధనలు

```
// BFS – queue. దగ్గరివి ముందు.
const bfs = (g,s,t) => { const d = new Array(g.length).fill(-1); d[s]=0;
  const q=[s]; for(let h=0;h<q.length;h++){ const u=q[h]; if(u===t) return d[u];
    for(const v of g[u]) if(d[v]<0){ d[v]=d[u]+1; q.push(v); } } return -1; };

// DFS – recursion. ఒక దారి పట్టుకుని చివరి వరకు.
const dfsFirst = (g,s,t) => { const seen=new Uint8Array(g.length);
  const go=(u,d)=>{ if(u===t) return d; seen[u]=1;
    for(const v of g[u]) if(!seen[v]){ const r=go(v,d+1); if(r>=0) return r; }
    return -1; };
  return go(s,0); };
```

రెండూ దారి కనుగొంటాయి. రెండూ ప్రతి node ని ఒక్కసారే చూస్తాయి. రెండూ $O(V+E)$.

కొలిచి చూద్దాం.

4. మొదటి విరుపు – DFS అతి చిన్న దారి ఇచ్చేది 18% సార్లు

అంచులు అన్నీ ఒకే బరువు • 17,528 graphs lo దారి ఉంది

పద్ధతి | సగటు దారి పొడవు | అతి చిన్నది ఇచ్చిన సార్లు

BFS	1.69	17,528 / 17,528 (100%)
DFS	5.00	3,234 / 17,528 (18%)

DFS తప్పు జవాబు ఇచ్చిన సార్లు : 14,294 (81.5%)

అత్యంత చెడ్డ తేడా : BFS 1 అడుగులు, DFS 11 (12 nodes)

సగటున DFS దారి 196% పొడవు

BFS 1 అడుగు, DFS 11 అడుగులు – 12 NODES ఉన్న GRAPH LO

లక్ష్యం నేరుగా పక్కనే ఉంది. DFS మొదటి పొరుగుని ఎంచుకుని, అక్కడి నుంచి లోతుగా వెళ్ళి, graph మొత్తం తిరిగి, చివరికి లక్ష్యాన్ని పొడవైన దారి ద్వారా చేరుకుంది.

మరియు ఇది ఒక అరుదైన సందర్భం కాదు – **81.5% సార్లు** DFS పొడవైన దారి ఇచ్చింది.

ఎందుకు BFS కి హామీ ఉంది - ఒక్క వాక్యంలో

BFS పొరలుగా విస్తరిస్తుంది: మొదట 1 అడుగులో ఉన్నవన్నీ, ఆపై 2 అడుగుల్లో ఉన్నవన్నీ.

కాబట్టి ఒక node ని మొదటిసారి చూసినప్పుడు, అంతకంటే తక్కువ అడుగుల్లో చేరే దారి ఉండటం అసాధ్యం - ఎందుకంటే ఆ పొరని మనం ఇప్పటికే పూర్తిగా చూశాం.

ఆ హామీకి ఆధారం: ప్రతి అంచా ఒకే బరువు. అంచులకి వేర్వేరు బరువులు ఉంటే "పొర" అనే ఆలోచనే చెల్లదు - మరియు అక్కడే Dijkstra మొదలవుతుంది.

Part 3 – రెండో విరుపు: ఋణాత్మక అంచులు

5. Step – బరువులు ఉంటే Dijkstra

```
function dijkstra(n, edges, s) {
  const d = new Array(n).fill(Infinity); d[s] = 0;
  const done = new Uint8Array(n);
  for (let k = 0; k < n; k++) {
    let u = -1, best = Infinity;
    for (let i = 0; i < n; i++) if (!done[i] && d[i] < best) { best = d[i]; u = i; }
    if (u < 0) break;
    done[u] = 1; // ← ఇక్కడే ఊహ
    for (const [v, w] of g[u]) if (d[u] + w < d[v]) d[v] = d[u] + w;
  }
  return d;
}
```

6. Step – Dijkstra నడిచే ఊహ

ఆ `done[u] = 1` అనే పంక్తి ఒక ప్రకటన: "`u` యొక్క దూరం ఇక ఎప్పటికీ మారదు."

ఆ ప్రకటన ఎందుకు నిజం – మరియు ఏ షరతు మీద

`u` అనేది మిగిలిన వాటిలో అతి తక్కువ దూరం ఉన్నది. ఇంకో దారి ద్వారా `u` కి చేరాలంటే, ముందు ఇంకో node `x` ద్వారా వెళ్ళాలి, మరియు $d[x] \geq d[u]$.

అప్పుడు ఆ కొత్త దారి పొడవు = $d[x] + (x$ నుంచి u కి బరువు).

ఆ బరువు ≥ 0 అయితే ఆ మొత్తం $d[x] \geq d[u]$ కంటే తక్కువ కాలేదు. కాబట్టి $d[u]$ తుది. ✓

ఆ బరువు ఋణాత్మకం అయితే – ఆ మొత్తం $d[u]$ కంటే తక్కువ కావచ్చు. మరియు మనం `u` ని ఇప్పటికే "పూర్తి" అని గుర్తుపెట్టేశాం. ✗

7. రెండో విరుపు – ఒక్క ఋణాత్మక అంచు

1,00,000 యాదృచ్ఛిక graphs · Dijkstra vs Bellman-Ford

అంచు బరువులు	పరీక్షించినవి	Dijkstra తప్పు	%
0..9 (ఋణాత్మకం లేదు)	1,00,000	0	0.0%
-1..9	94,816	269	0.3%
-3..9	75,233	871	1.2%

ఒక ఉదాహరణ (6 nodes):

అంచులు: 0→1(4) 0→4(4) 1→5(6) 2→0(4) 3→0(9) 3→1(-3)
4→3(-3) 4→5(6) 5→1(6) 5→2(8) 5→4(6)
node 2 కి - Dijkstra 18, నిజం 12

DIJKSTRA 18 అంది. నిజమైన దూరం 12. CRASH లేదు, హెచ్చరిక లేదు.

అసలు దారి 0 → 4 → 3 → 1 → 5 → 2 గుండా వెళ్తుంది, మరియు అందులో **రెండు ఋణాత్మక అంచులు** ఉన్నాయి (4→3 మరియు 3→1).

Dijkstra 1 ని ముందే "పూర్తి" అని గుర్తుపెట్టింది (దూరం 4), ఆ తర్వాత 4→3→1 ద్వారా దూరం **-2** కి తగ్గుతుందని తెలిసినా దాన్ని **తిరిగి తెరవలేదు**.

వేగాన్ని ఇచ్చిన ఆ ఒక్క ఊహ తప్పు జవాబునే ఇచ్చింది.

మరియు గమనించండి - 98.8% సార్లు అది సరైనదే

ఋణాత్మక అంచు ఉన్నా, అది **అతి చిన్న దారిలో** ఉండకపోతే జవాబు సరిగ్గానే వస్తుంది.

కాబట్టి మీరు ఐదు test cases నడిపి "పనిచేస్తోంది" అని అనుకుంటారు.

Deep 01 lo 83%, Deep 03 lo 99.3%, Deep 04 lo 52.5% - ఇప్పుడు 98.8%. ఇదే ఈ series యొక్క కేంద్ర నమూనా.

8. Step – Bellman-Ford • మరియు ఋణాత్మక వలయాలు

```
function bellmanFord(n, edges, s) {
  const d = new Array(n).fill(Infinity); d[s] = 0;
  for (let i = 0; i < n - 1; i++) // n-1 - ఏ దారి అంతకంటే పొడవుగా ఉండదు
    for (const [u, v, w] of edges)
      if (d[u] !== Infinity && d[u] + w < d[v]) d[v] = d[u] + w;
  return d;
}
const hasNegativeCycle = (n, edges, d) =>
  edges.some(([u, v, w]) => d[u] !== Infinity && d[u] + w < d[v]);
```

Bellman-Ford **ఏ ఊహ చేయదు**. అది కేవలం అన్ని అంచులనే పదే పదే సడలిస్తుంది.

ఎందుకు సరిగ్గా $N-1$ సార్లు

n nodes ఉన్న graph lo, ఒక వలయం లేని దారిలో గరిష్టంగా $n-1$ అంచులు ఉంటాయి.

ప్రతి round **కనీసం ఒక అంచు** విలువని సరిచేస్తుంది. కాబట్టి $n-1$ rounds తర్వాత అన్ని దారులూ పూర్తిగా సడలించబడ్డాయి.

మరియు $n-1$ తర్వాత ఇంకా ఏదైనా సడలించు సాధ్యమైతే – ఆ దారిలో n కంటే ఎక్కువ అంచులు ఉన్నాయి, అంటే అందులో ఒక **వలయం** ఉంది, మరియు అది దూరాన్ని తగ్గిస్తోంది కాబట్టి అది **ఋణాత్మక వలయం**.

కాబట్టి ఆ ఒక్క అదనపు round **ఒక పరీక్ష** – §12 lo దాన్ని $n-2$ గా మార్చితే **147/150** విఫలం.

Part 4 – మూడో విరుపు: సరైన జవాబు, తప్పు ఖర్చు

9. Step – "visited" ఎక్కడ గుర్తుపెట్టాలి

BFS లో ఒక చిన్న ఎంపిక ఉంది, మరియు రెండూ "పనిచేస్తాయి":

```
// (అ) queue లో పెట్టేటప్పుడు గుర్తుపెట్టడం
for (const v of g[u]) if (!seen[v]) { seen[v]=1; q.push(v); }

// (ఆ) queue నుంచి తీసేటప్పుడు గుర్తుపెట్టడం
const u = q[h]; if (seen[u]) continue; seen[u]=1;
for (const v of g[u]) if (!seen[v]) q.push(v);
```

రెండూ **అవే nodes ని సందర్శిస్తాయి**, అవే దూరాలు ఇస్తాయి. జవాబు ఒకటే.

10. మూడో విరుపు – 59.8 రెట్లు ఎక్కువ పని, అదే జవాబు

BFS • "visited" ఎప్పుడు గుర్తుపెట్టాలి – queue లో పెట్టేటప్పుడా, తీసేటప్పుడా?

nodes	సాంద్రత	push అప్పుడు	pop అప్పుడు	ఎన్ని రెట్లు ఎక్కువ pushes
50	0.1	50	125	2.5×
200	0.1	200	2,002	10.0×
500	0.1	500	12,507	25.0×
200	0.3	200	5,967	29.8×
200	0.6	200	11,955	59.8×

200 NODES • QUEUE LO 200 కి బదులు 11,955 ENTRIES

"pop అప్పుడు గుర్తుపెట్టడం" అంటే ఒక node ని **దాని వ్రతి పొరుగు** queue లో పెట్టగలదు – అది ఇప్పటికే queue లో ఉన్నా.

కాబట్టి queue పరిమాణం **nodes సంఖ్య కాదు, అంచుల సంఖ్య** అవుతుంది. సాంద్రత పెరిగే కొద్దీ తేడా పెరుగుతుంది: $2.5 \times \rightarrow 59.8 \times$.

సమయం ఇప్పటికీ $O(V+E)$ – కాబట్టి Big-O మారలేదు. మారినది **memory**, మరియు అది E కి అనులోమానుపాతంలో.

ఈ విరుపుకి ఒక నియమం రాయలేము - మరియు అది ఒక పాఠం

§12 యొక్క fuzz lo ఈ మార్పు **0/150** చూపిస్తుంది, మరియు అది **సరైన ఫలితం**: జవాబు నిజంగా మారదు.

కాబట్టి దీనికి సాక్ష్యం ఒక invariant కాదు - **పై పట్టిక**. సరైనతనాన్ని నియమాలు కాపాడతాయి; **ఖర్చుని కొలతలు మాత్రమే బయటపెడతాయి**.

(ఈ series lo ఇదే నమూనా HLD docs lo పదే పదే వచ్చింది - "ఏదీ తప్పు జరగలేదు" నియమాలు load సమస్యలని పట్టుకోలేవు.)

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code

```
// BFS – అంచులు అన్నీ ఒకే బరువు అయితేనే అతి చిన్న దారి (§4)
function bfsDist(n, adj, s) {
  const d = new Array(n).fill(Infinity); d[s] = 0;
  const q = [s];
  for (let h = 0; h < q.length; h++) {
    const u = q[h];
    for (const v of adj[u])
      if (d[v] === Infinity) { d[v] = d[u] + 1; q.push(v); } // *push* అప్పుడు గుర్తు (§10)
  }
  return d;
}
```

bfsDist lo q ని ఎప్పుడూ shift() చేయలేదని గమనించండి – ఒక h సూచిక వాడాను.
Array.shift() పెద్ద queue మీద O(n), కాబట్టి అది BFS ని రహస్యంగా O(V²) చేస్తుంది.

12. 16,000 graphs • Floyd-Warshall తో పోలిక • mutation testing

ఏడు నియమాలు, అన్నీ స్వతంత్ర Floyd-Warshall తో పోల్చినవి: ఋణాత్మక వలయ గుర్తింపు సరైనది, Bellman-Ford = Floyd-Warshall, త్రిభుజ అసమానత (ప్రతి అంచుకీ $d[v] \leq d[u] + w$), బరువులు ≥ 0 అయితే Dijkstra = Bellman-Ford, BFS = అన్ని బరువులు 1 అయినప్పటి Bellman-Ford, $d[s] = 0$ మరియు చేరలేనివి Infinity, BFS దూరాలు 0..n-1 పరిధిలో.

400 runs • 16,000 graphs • 89,670 అంచులు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

graphs	16,000
negEdges	6,349
negCycles	1,003
unreachable	12,177
chains	5,535

అ CHAINS 5,535 - నా DATA LO ఒక రంధ్రాన్ని పూడ్చడానికి

మొదట నా graphs అన్న యాదృచ్ఛిక దట్టమైనవి. అప్పుడు Bellman-Ford ని $n-1$ నుంచి $n-2$ rounds కి తగ్గించే mutation **0/150** చూపించింది.

కారణం: దట్టమైన graph lo అతి చిన్న దారులు **2-3 అంచులు** మాత్రమే, కాబట్టి $n-2$ rounds సరిపోతాయి.

గొలుసు (chain) graphs జోడించాక - $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n-1$, అతి చిన్న దారికి సరిగ్గా $n-1$ అంచులు - అదే mutation **147/150**.

Deep 01 మరియు Deep 03 లోని అదే పాఠం, మూడోసారి: ఒక mutation బతికితే, ముందు మీ data ని అనుమానించండి.

==== mutation testing ====

మార్పు లేని code	→ 0/400 విఫలం
BFS lo visited ని push కి బదులు pop అప్పుడు	→ 0/150 విఫలం
BFS lo queue కి బదులు stack (pop చివరి నుంచి)	→ 107/150 విఫలం
ఉదా: bfs[2] = 3, unit-bellman = 2	
Dijkstra lo 'done' తనిఖీ తీసేస్తే	→ 150/150 విఫలం
ఉదా: dijkstra[2] = Infinity, bellman = 9	
Dijkstra lo కనిష్ఠానికి బదులు గరిష్ఠం ఎంచుకుంటే	→ 150/150 విఫలం
ఉదా: dijkstra[1] = Infinity, bellman = 3	
Bellman-Ford lo $n-1$ కి బదులు $n-2$ సార్లు	→ 147/150 విఫలం
ఉదా: ఋణాత్మక వలయం: మనం true, Floyd false	
Bellman-Ford lo <code>d[u] != Infinity</code> తనిఖీ తీసేస్తే	→ 0/150 విఫలం
ఋణాత్మక వలయ తనిఖీలో <code><</code> ని <code><=</code> గా	→ 150/150 విఫలం

రెండో వరుస ముఖ్యమైనది: **queue ని stack గా మార్చడం** అంటే BFS ని DFS గా మార్చడం - మరియు అది **107/150** విఫలం, సరిగ్గా §4 lo కొలిచిన కారణంతో.

మరియు నా నియమం ఒకటి తప్పు - 400 LO 106 విఫలం

"ఋణాత్మక వలయ గుర్తింపు Floyd-Warshall తో ఏకీభవించాలి" అని రాశాను. Base code **106/400** విఫలమైంది.

Code తప్పు కాదు. **రెండు వేర్వేరు విషయాలు కొలుస్తున్నాయి:** single-source Bellman-Ford **source నుంచి చేరగలిగే** ఋణాత్మక వలయాలనే చూస్తుంది; Floyd-Warshall **graph lo ఎక్కడ ఉన్నా** చూస్తుంది.

నియమాన్ని "చేరగలిగే వలయాలకే" పరిమితం చేశాక - 0/400.

ఇది కూడా ఒక సాధారణ తప్పు: రెండు సరైన answers వేర్వేరు ప్రశ్నలకి జవాబులు కావచ్చు.

13. బతికిన రెండు మార్పులు

1. visited ని pop అప్పుడు గుర్తుపెట్టడం - §10 lo వివరించినట్టు, జవాబు నిజంగా మారదు. సాక్ష్యం §10 యొక్క పట్టిక (59.8×), ఒక నియమం కాదు.

2. `d[u] !== Infinity` తనిఖీ తీసేయడం – ఇది ఇంకా ఆసక్తికరమైనది:

```
JavaScript lo:
Infinity + 5      = Infinity
Infinity + 5 < 10 = false → సడలింపు జరగదు, తనిఖీ అనవసరం

C++ / Java lo INT_MAX ని sentinel గా వాడితే (32-bit):
INT_MAX + 5      = -2147483644 ← ఋణాత్మకం అయింది (overflow)
-2147483644 < 10 = true → తప్పుడు సడలింపు జరుగుతుంది
```

ఈ తనిఖీ **JAVASCRIPT LO** నిజంగా అనవసరం – మరియు **C++/JAVA LO** తప్పునిసరి

JS lo `Infinity` ఒక నిజమైన విలువ, మరియు దానికి ఏమి కలిపినా `Infinity`. కాబట్టి "చేరని node నుంచి సడలించడం" తనంతట తానే ఆగిపోతుంది.

C++ lo `INT_MAX` ఒక **నటన** – అది ఒక సాధారణ సంఖ్య, మరియు దానికి కలిపితే **overflow** అవుతుంది, ఫలితం ఋణాత్మకం, మరియు అది **అన్ని దూరాలనీ పాడు చేస్తుంది**.

DSA Deep 01 lo `hi = a.length` – అదే ఆకారం. JS ఒక bug ని కప్పిపుచ్చుతుంది; వేరే భాషలో అదే code విరుగుతుంది.

కాబట్టి ఆ పంక్తిని **ఉంచుతున్నాను**, మరియు అది నా భాషలో అనవసరమని చెబుతున్నాను – ఎందుకంటే మీరు దీన్ని Java lo రాస్తే అది అవసరం.

14. Follow-ups – A*, 0-1 BFS, topological sort

"లక్ష్యం ఎక్కడ తెలిస్తే Dijkstra ని వేగవంతం చేయొచ్చా?"

A*: `d[u]` కి బదులు `d[u] + h(u)` ప్రకారం ఎంచుకోవడం, ఇక్కడ `h(u)` అనేది `u` నుంచి లక్ష్యానికి **అంచనా** దూరం. షరతు: `h` ఎప్పుడూ **తక్కువ అంచనా** వేయాలి (admissible) – లేకపోతే A* అతి చిన్న దారిని **మిస్ చేస్తుంది**, సరిగ్గా §7 యొక్క అదే కారణంతో. నేను A ని కొలవలేదు.*

"అంచు బరువులు 0 లేదా 1 మాత్రమే అయితే?"

0-1 BFS: heap అవసరం లేదు, ఒక **deque** చాలు – బరువు 0 అంచుని ముందు చేర్చడం, బరువు 1 ని వెనక. $O((V+E) \log V)$ కి బదులు **$O(V+E)$** . ఇది BFS యొక్క "పొర" ఆలోచనని బరువులున్న graph కి విస్తరించడం.

"ఆధారాల క్రమం (dependencies)?"

Topological sort – DFS యొక్క నిజమైన బలం. ఇక్కడ DFS **సరైన సాధనం**: ఒక node యొక్క అన్ని పిల్లలూ పూర్తయ్యాక దాన్ని జాబితాలో చేర్చడం, చివర్లో తిరగేయడం. మరియు DFS lo ఒక వలయం కనిపిస్తే – **ఆధారాలు వృత్తాకారంలో ఉన్నాయి**, topological sort ఉనికిలో లేదు.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **A*** (ఫైన), **heap తో Dijkstra** (నా అమలు $O(V^2)$ – ఇది చిన్న graphs కి సరైనది, కానీ పెద్దవాటికి priority queue కావాలి, మరియు ఆ crossover ని నేను కొలవలేదు), మరియు **topological sort**

15. ఏమి నేర్చుకున్నాం

1. DFS అతి చిన్న దారి కోసం కాదు. 18% సార్లు మాత్రమే సరైనది, సగటున దారి **196% పొడవు**, అత్యంత చెడ్డ సందర్భంలో **1 అడుగుకి బదులు 11**.

2. BFS యొక్క హామీ "ప్రతి అంచూ ఒకే బరువు" అనే దాని మీద ఆధారపడినది. ఆ ఆధారం పోతే "పొర" అనే ఆలోచనే చెల్లదు.

3. Dijkstra యొక్క వేగమే దాని బలహీనత. "ఈ node పూర్తయింది" అనే ఊహ బరువులు ≥ 0 అయితేనే నిజం – **1.2% తప్పు**, మరియు 18 vs 12.

4. మరియు అది 98.8% సార్లు సరైనదే. ఈ series lo నాలుగోసారి ఆ నమూనా: 83%, 99.3%, 52.5%, ఇప్పుడు 98.8%.

5. Bellman-Ford lo $n-1$ ఒక ఏకపక్ష సంఖ్య కాదు. ఇది "ఏ వలయరహిత దారికీ గరిష్టంగా $n-1$ అంచులు" అనే నిరూపణ – మరియు **n వ round ఒక పరీక్ష**.

6. అన్ని bugs జవాబుని మార్చవు. `visited` ని ఒక పంక్తి అలస్యంగా గుర్తుపెడితే queue **59.8 రెట్లు** పెద్దది, జవాబు అదే. దీనికి నియమం రాయలేము – **కొలత మాత్రమే**.

7. ఒక mutation బతికితే ముందు data ని అనుమానించండి. దట్టమైన graphs lo $n-2$ rounds సరిపోతాయి. గొలుసులు జోడించాక 0/150 → **147/150**.

8. రెండు సరైన జవాబులు వేర్వేరు ప్రశ్నలకి కావచ్చు. నా ఋణాత్మక-వలయ నియమం **106/400** విఫలమైంది – code తప్పు కాదు, "చేరగలిగే వలయాలు" vs "అన్ని వలయాలు".

9. Infinity ఒక బహుమతి. C++ lo `INT_MAX + w` overflow అవుతుంది; JS lo అది ఇంకా `Infinity`. **అదే తనిఖీ ఒక భాషలో అనవసరం, ఇంకొక భాషలో తప్పనిసరి.**

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ మూడు algorithms వేగం క్రమంలో ఉన్నాయి – మరియు అదే వాటి ఊహల బలం క్రమం.

BFS అత్యంత వేగం, మరియు అత్యంత బలమైన ఊహ (అన్ని అంచులూ సమానం).

Dijkstra మధ్యస్థం, మధ్యస్థ ఊహ (అంచులు ≥ 0).

Bellman-Ford అత్యంత నెమ్మది, మరియు ఏ ఊహ చేయదు.

కాబట్టి "ఏ algorithm" అనే ప్రశ్న నిజానికి **"నా data ఏ హామీ ఇస్తోంది"** అనే ప్రశ్న. మీరు ఇవ్వగల హామీ ఎంత బలమైనదో, అంత వేగవంతమైన algorithm వాడొచ్చు.

మరియు లేని హామీని ఊహించుకుంటే – **crash కాదు, తప్పు సంఖ్య.**